

OSを動かせるRISC-VのCPUを書く順序を考えた

2025/8/9 Kernel/VM探検隊@東京 No18 kanataso

<https://github.com/nananapo>

目的

- ・OSを動かせる程度のCPUを実装(RTLを記述)したい
- ・楽に書きたい。楽 = 下記の通り

提案する実装順序の利点

- ・各ステップのコード変更量が少ない
- ・ある機能を実装するとき、それ以外の機能を中途半端に実装する必要がない

実装開始時の注意点

- できるだけ数値をパラメータ化する
 - ・仕様に定義されている値
 - ・例: XLEN、ILENなど
 - ・マイクロアーキテクチャ、特にメモリに関連する数値
- 理由: 後から変えるのが非常に難しい (面倒になる)



実装にオススメのハードウェア記述言語 - Veryl

- ・SystemVerilogよりもモダン、書きやすい構文
- ・モジュール、パッケージ、関数のパラメータ化
- ・インターフェースの相互接続、関数の引数化が容易

提案する実装順序

①基本整数命令セット

RV32I (32ビット)



RV64I (64ビット)

まず、最も単純なISAであるRV32Iを実装する

XLEN(ビット幅)を動的・静的に変更したい場合は、基本整数命令セットを作る時点で考慮しておく(と楽(CSR、レジスタ、デコーダの変更が面倒))

①ポイント

- ・FPGAに合成する場合は、合成できる状態を維持しながら実装する
- ・主要な機能ごとにモジュール化する
 - ・例: レジスタファイル、パイプラインステージ

②実装のテスト

MMIO



riscv-arch-test



Zicsr (CSR、例外)



riscv-tests

利用するテストスイート

riscv-tests

- ・メモリを介してホストと通信、テスト成否の報告
- ・CSRの読み書き機能、簡単な例外機構が必要

riscv-arch-test

- ・riscv-testsよりもテスト数が多い
- ・メモリに結果を書き込む→ダンプする機能が必要

riscv-arch-testの結果はMMIOがあればわかるので、先にriscv-arch-testでテストする。そのあとriscv-testsを実行できる程度のCSR、例外を実装してテストする。

riscv-arch-testによるテスト方法: <https://kanataso.hatenablog.jp/entry/2024/12/13/145927>

デバッグ手法の例: <https://www.docswell.com/s/2700235/Z8G797-makecpu-3-1t>

②ポイント

- ・任意のC、アセンブリを1つのコマンドで実行できる仕組みを整えておく
- ・GitHub Actionsなどで自動でテストするようにするとデバッグが容易になる
- ・機能、拡張を実装したら必ずテストで確かめる(riscv-testsもriscv-arch-testも仕様を全然網羅していないので、実装した機能のテストを自分で書く)
- ・Konata (パイプライン可視化ツール)を使えるようにしておく、バグの発生個所が爆速で分かる
- ・既存のISS (命令セットシミュレータ)と比較できるようにすると良い (Spikeなど)

③ベンチマーク

M拡張
(乗算、除算、剰余)



CoreMark
(ベンチマーク)

- ・基本的な四則演算、メモリ操作ができるようになるとベンチマークプログラムを動かせるようになる。
- ・M拡張を実装するとき、拡張の有効無効を静的・動的に切り替えたい場合はそれを考慮して実装する(主にmisaレジスタで動的に変更可)

③ポイント

- ・コミット時に自動で実行すると、変更の性能への影響が分かってうれしい

④アトミック操作

A拡張

- ・シングルコア、インオーダー実行なら複雑な実装は不要
- ・マルチコア、アウトオブオーダー実行なら要検討

④ポイント

- ・経験則だが、だんだんとリッチな構成にしていくよりも、どのような構成にするかを最初から決めておいてそれに沿った実装をする方がよさそう

⑤特権レベルの実装

M-mode
(CSRの実装)



U-mode



S-mode

(トラップの委譲・ページング)

特権レベルの種類

高い ⇄ 低い

M-mode > S-mode > U-mode

実装の選択肢

- M : 特権の必要ないソフトウェア
- M、U : システムソフトウェアをM-modeで動かしU-modeでアプリケーションを動かす

M、U、S: 汎用OSを動かす

ページングの動作確認にはSvxxVisualizerがオススメ

・ <https://github.com/Alignof/SvxxVisualizer>

⑥割り込み

CLINT: ソフトウェア割り込み、タイマ割り込み

PLIC: 外部割り込みと制御

具体的な実装順序の例

実装の解説: Verylで作るCPU

<https://cpu.kanataso.net/index.html>

実装の例: bluecore

<https://github.com/nananapo/bluecore>

Linuxを起動できるまでゼロから実装する方法を書いたのでRISC-VでCPUを実装したい人はどうか参考にしてください
m(_ _)m



<https://cpu.kanataso.net/>



Linux

- ・Device Tree Source(dts)を書いてメモリに配置する
- ・Supervisor Binary Interface(SBI)経由で起動する(例: OpenSBI)

xv6

- ・virtioデバイスの実装が必要
 - ・RV32Iで動かすなら移植する(CSR、ELFのパース処理変更など)
- どちらもC拡張(圧縮命令)は不要(に設定できる)

⑦OSを動かす